

HobbyFi Copilot

AI Assistant for the Vendor / Partner Portal — Architecture & Design Report

Prepared for: HobbyFi Hiring Process | Scope: partner.hobbyfi.in Copilot | July 2026

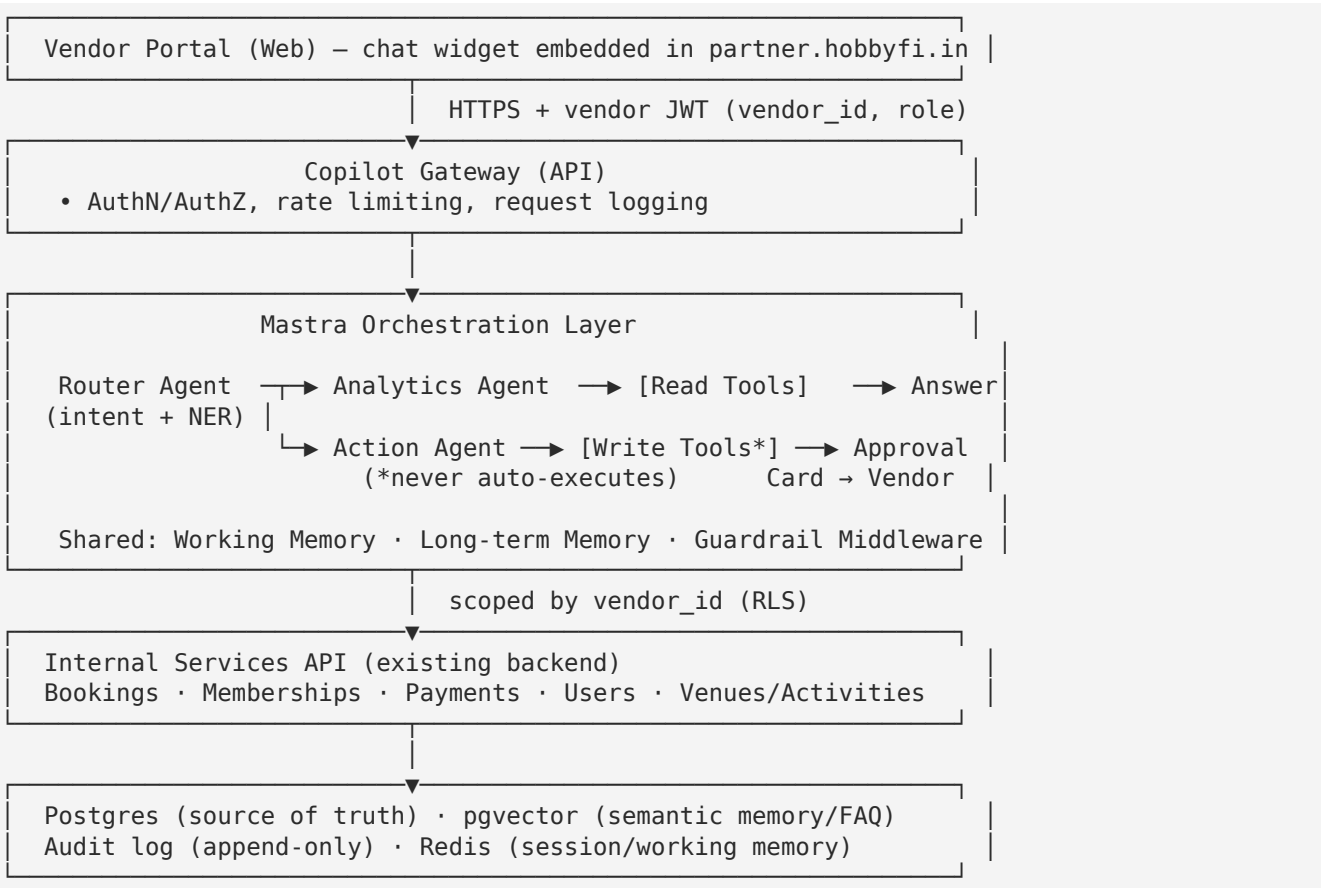
1. Problem Framing

HobbyFi's partner portal (partner.hobbyfi.in) serves vendors who run courts, studios and classes across Play, Pass, Swipe and Community. Vendors currently have to click through dashboards to answer simple questions ("what did I earn today?") and to perform simple edits (extend a trial, fix a membership date). The Copilot's job is to collapse that into a chat interface that (a) answers analytical questions grounded in real vendor data, and (b) proposes and — only on explicit vendor approval — executes write actions. Two design principles drive everything below:

- **Read is cheap, write is expensive.** Read queries can be answered autonomously; any state-changing action must pass through a human-in-the-loop approval gate, no exceptions.
- **The LLM never touches the database directly.** It only calls typed, permissioned tools that wrap existing/planned backend APIs. This keeps SQL injection, cross-vendor leakage and runaway writes structurally impossible, not just prompt-discouraged.

2. Architecture Overview

Layered view, vendor portal → copilot service → data:



Everything below the 'Internal Services API' line already exists (or is a natural extension of) HobbyFi's product data — bookings, memberships/passes, payments, venues, users. The Copilot adds three new components: the Mastra orchestration layer, a thin audit/approval service, and a vector store for semantic memory / FAQ retrieval.

3. Tools & Frameworks

Layer	Choice	Why
Agent orchestration	Mastra	Already the team's chosen stack — native TypeScript, first-class Workflows (typed, resumable, step-level retries) and Tools that map cleanly onto our existing REST/GraphQL services. Also ships built-in memory primitives (thread + working + semantic) we reuse instead of hand-rolling.
Reasoning model	Claude (Sonnet-tier) for the Analytics & Action agents	Strong function-calling reliability and instruction-following for constrained, tool-only responses; good at refusing to answer without a grounding tool call when guarded properly.
Routing / intent model	Claude (Haiku-tier)	Cheap, low-latency classification of read-vs-write intent and slot extraction (entity, date range, activity) before handing off to the heavier agent.
Tool transport	Typed Mastra Tools → internal REST/GraphQL	No free-form SQL generation. Each tool has a Zod schema; the model can only call functions that were explicitly exposed, with parameter validation before any network call.
Structured data	Postgres (existing)	Vendor, user, booking, membership, transaction data — source of truth, queried only via services, never directly by the LLM.
Semantic memory / FAQ	pgvector (or Pinecone)	Embeds resolved past queries, vendor-specific notes, and HobbyFi policy docs (refund rules, trial caps) for retrieval-augmented answers on 'how do I / what's the policy on...' questions.
Session/working memory	Redis	Fast, TTL-based store for entity references and in-flight conversation state (see §4).
Observability	Mastra tracing + Langfuse (or similar)	Full trace of router decision → tool calls → args → results → final answer, for debugging and audits.

4. Memory Strategy

Memory is split into four scopes, each with a distinct lifetime and purpose — this avoids the common failure mode of one giant context blob that leaks stale or cross-vendor data.

- **Working memory (per conversation, Redis, TTL ~30 min):** Resolves anaphora like "this user" or "that trial" — the last entity returned by a tool call (`user_id`, `membership_id`, `booking_id`) is cached and re-injected so follow-up turns don't need re-identification. Cleared on session end or vendor switch.
- **Thread memory (per conversation, Mastra built-in):** Rolling window of recent turns (summarized once it exceeds ~15 turns) so the agent keeps conversational coherence without re-sending the full history to the model every call.
- **Long-term vendor memory (per vendor_id, Postgres):** Durable, low-volume facts the vendor has told the Copilot or that are inferable from usage — default activity filters, preferred report format, recurring questions. Never stores anything sensitive about end-users, only vendor operating preferences, and is editable/erasable by the vendor from settings.
- **Semantic memory (pgvector, cross-conversation):** Embeddings of past (query → tool-plan → outcome) triples, used as few-shot retrieval to improve tool selection on ambiguous phrasing (e.g. "trial users" vs "free-trial bookings"), plus embedded policy/FAQ docs for RAG answers. This is retrieval for the agent's own accuracy, not a substitute for the write-tool grounding rule in §5.

Explicitly excluded from memory: raw end-user PII beyond what a tool call needs in the moment, and any write-approval decision is logged to the immutable audit trail (§5) rather than to conversational memory — approvals must be re-confirmed each time, never auto-approved from a remembered preference.

5. Guardrails Framework

5.1 Structural (can't-be-prompted-around)

- Row-level scoping: every tool call is executed server-side with `vendor_id` from the authenticated session, not from anything the model outputs — a vendor's Copilot literally cannot query another vendor's data.
- Tool-only data access: no SQL/DB credentials are ever in the model's context or reachable by it; only the pre-approved tool catalog is invocable.
- Read/write separation at the tool-registry level: write tools are registered with `executeMode: "proposal-only"` in Mastra — calling one returns a diff/preview object, never performs the mutation. A second, separate privileged call (triggered only by the vendor clicking Approve in the UI, never by the LLM) performs the actual write.

5.2 Business-rule validation (before an action can even be proposed)

- Zod-schema parameter validation on every tool (types, ranges, enums).
- Domain caps, e.g.: trial extensions capped at 30 days per action / 60 days cumulative; membership end-date can't move to the past; discounts/refunds above a threshold require a second approver (owner/admin role, not the same vendor user).
- Idempotency keys on all write tools so a retried or double-clicked approval can't double-apply.

5.3 Human-in-the-loop approval

- Every write intent renders an Approval Card: plain-language summary, structured before → after diff, and the exact tool + params that will run.
- Vendor can Approve, Edit-then-approve, or Reject; edits re-validate against the same business rules before a new diff is shown.
- Approval Cards expire (e.g. 15 min) to avoid stale actions being approved against since-changed data.

5.4 Answer-grounding & anti-hallucination

- System prompt + a post-generation checker require every numeric or factual claim in a read answer to trace back to a tool result in that turn; ungrounded numeric claims are stripped and replaced with "I don't have that figure — want me to pull it?"
- Tool results are wrapped and labelled as data, not instructions, so text inside them (e.g. a user's free-text note) can't be interpreted as new commands — mitigates prompt injection via stored user content.

5.5 Audit & recovery

- Append-only `audit_log` table: every proposal, approval/rejection, executing tool, before/after state, actor, and timestamp — queryable by vendor and by HobbyFi ops for compliance.
- Reversible actions (date changes, trial extensions) get a one-click Undo for a limited window, backed by the same audit trail.

6. Workflow Orchestration

Implemented as a Mastra Workflow with typed steps, so each stage can be traced, retried, and short-circuited independently:

- **1. Intake & Router:** Haiku-tier classification splits every message into read (analytics) vs write (action) intent, plus slot extraction (entity refs, date ranges, activity/venue). Ambiguous or multi-intent messages are decomposed into sub-tasks and run sequentially.
- **2. Analytics Agent (read path):** Plans which read tool(s) to call (e.g. `getRevenue`, `listTrialUsers`, `listBookings`), calls them in parallel where independent, aggregates/formats the result (table, single figure, or chart spec), and returns an answer with a "data as of <timestamp>" footer.
- **3. Action Agent (write path):** Resolves the target entity (using working memory for pronouns), calls the corresponding proposal-only write tool, runs business-rule validation, and renders the Approval Card — then

halts. Execution only resumes on an explicit vendor Approve event from the frontend, which triggers the privileged execute step.

- **4. Execution & Confirmation:** On approval, the workflow calls the real write endpoint, writes to the audit log, and posts a confirmation message back into the chat ("Done — trial for Aisha K. extended to 30 Jul.").
- **5. Error/Fallback handling:** Tool failures trigger a bounded retry (exponential backoff, max 2 retries) before falling back to a plain-language error and, for write actions, an offer to escalate to HobbyFi support.
- **6. Async/long-running queries:** Heavy aggregations (e.g. "export last quarter's bookings by venue") are dispatched as a background Mastra workflow run; the vendor gets an in-chat notification with a download link when it completes, instead of blocking the conversation.

7. Mock Data Schema

Minimal relational schema sufficient to back the tool catalog below (existing HobbyFi entities: Play/Pass activities, bookings, memberships, users, vendors).

Table	Key Columns
vendors	vendor_id (PK), name, city, category, status
venues	venue_id (PK), vendor_id (FK), name, address, activities[]
activities	activity_id (PK), venue_id (FK), name (e.g. Badminton), type [court class]
users	user_id (PK), name, phone, email, city, joined_at
memberships	membership_id (PK), user_id (FK), vendor_id (FK), activity_id (FK), plan_type, status [trial active expired], start_date, end_date
bookings	booking_id (PK), user_id (FK), venue_id (FK), activity_id (FK), slot_date, slot_time, amount, split_count, status
transactions	txn_id (PK), booking_id / membership_id (FK), vendor_id (FK), amount, method, status, created_at
copilot_audit_log	action_id (PK), vendor_id, actor_user, intent_type, tool_name, params_json, before_state, after_state, approval_status, created_at

Representative tool catalog exposed to the agents:

Tool	Mode	Example params
getRevenue	read	vendor_id, date_range
listTrialUsers	read	vendor_id, activity_id?
listBookings	read	vendor_id, venue_id?, date_range?
updateMembershipDate	write (proposal-only)	membership_id, new_end_date
extendTrial	write (proposal-only), capped	membership_id, extra_days (≤30)
issueRefund	write (proposal-only), dual-approve above ₹X	transaction_id, amount, reason

8. Example Walkthroughs

- **Read: "What is my revenue today?"** → Router tags read/analytics → Analytics Agent calls `getRevenue(vendor_id, date=2026-07-12)` → formats a single figure with a same-day comparison → "₹18,400 today, up 12% vs last Sunday. (as of 6:42 PM)". No approval needed, nothing written.
- **Write: "Increase Rohit's free trial by a week"** → Router tags write → Action Agent resolves "Rohit" via working memory / a disambiguation prompt if multiple matches → calls `extendTrial(membership_id, extra_days=7)` in proposal mode → renders an Approval Card ("Extend Rohit Kumar's Badminton trial: 18 Jul → 25 Jul") → vendor taps Approve → execute step runs the real update, logs to `audit_log`, confirms in chat.

9. Why This Design

- Matches the current stack (Mastra) instead of introducing a parallel framework, minimizing integration risk.

- Separates "can the model say it" from "can the model do it" — writes are structurally gated outside the LLM's control, not just prompted to be careful.
- Scales cleanly: new read tools (more report types) or write tools (more actions) are additive to the tool catalog and inherit the same approval/audit machinery for free.
- Memory is scoped and TTL'd so the assistant feels context-aware within a session without becoming a growing liability across sessions or vendors.